

Relatório de implementação do Chat P2P

Grupo 11:

Daniel Carvalho Mendes (231012737);

Fernanda Marques Pereira (222024560);

Thiago Carvalho Silva (222006463).

*Link para o repositório do projeto no **GitHub**:*

[P2P-chat / Grupo 11 - GitHub](#)

1.0 INTRODUÇÃO

A arquitetura tradicional de comunicação na internet é historicamente fundamentada no modelo cliente-servidor, onde todas as interações e mensagens precisam passar obrigatoriamente por um nó central. Embora prático e amplamente adotado, esse modelo cria pontos únicos de falha na infraestrutura e concentra o controle do tráfego de dados nas mãos de intermediários.

A arquitetura do cliente *Peer-to-Peer* (P2P) foi projetada com um foco rigoroso em modularidade e separação de responsabilidades, atuando como contra-fluxo. Desenvolvido em Python, o sistema é estruturado em camadas lógicas que vão desde a inicialização via *main.py* até a interação com entidades externas (usuários, outros *peers* via TCP e o Servidor de *Rendezvous*).

O núcleo da aplicação opera através de um orquestrador principal *p2p_client.py*, que atua gerenciando o ciclo de vida do programa e coordenando uma série de sub módulos independentes responsáveis por tarefas específicas. O diagrama presente na Figura 1 ilustra a arquitetura de software do cliente *Peer-to-Peer* altamente modularizado. O design do sistema é focado em gerenciar conexões de rede, comunicação com servidores de descoberta e a interação direta com o usuário, dividindo o fluxo de trabalho em quatro camadas lógicas que garantem uma clara separação de responsabilidades.

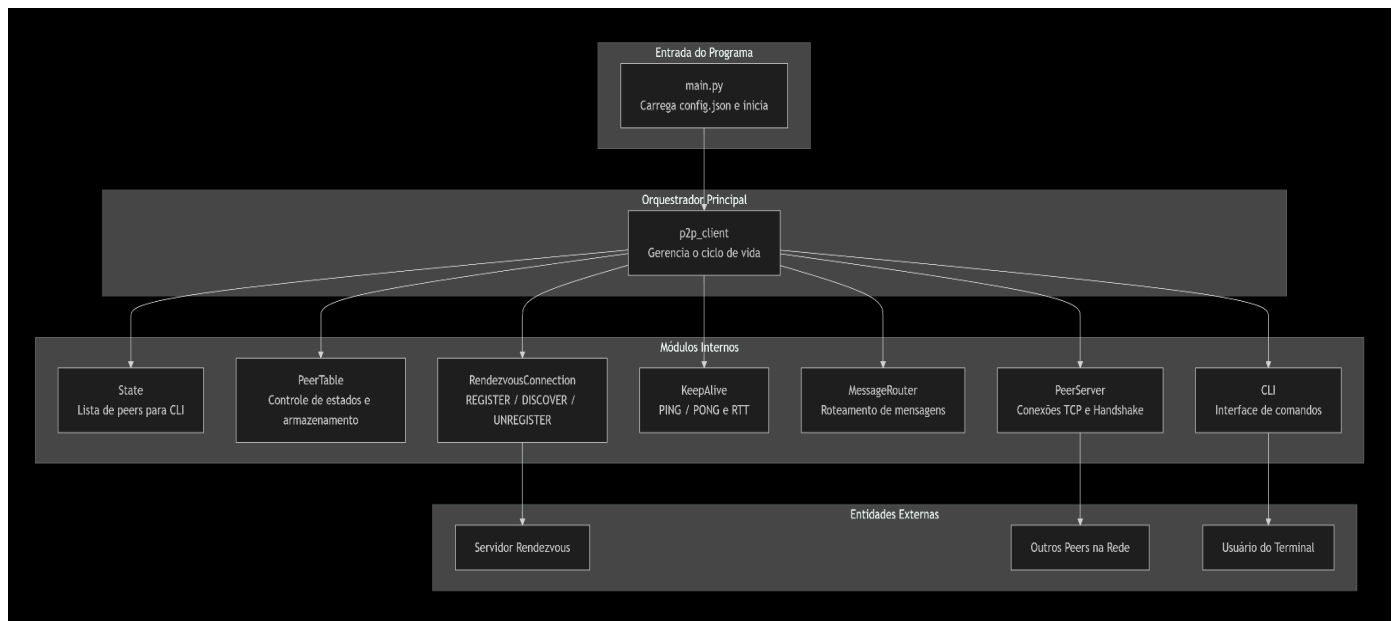


Figura 1. Diagrama de arquitetura do projeto.

A aplicação flui a partir de *main.py*, partindo da inicialização até a interação com agentes externos. A função do script primário é carregar as configurações do sistema a partir das predefinições contidas em *config.json* e iniciar a execução da aplicação.

Daí, vamos para o componente *p2p_client.py*, o qual atua como o gerenciador da aplicação de software. O mesmo é responsável por gerenciar todo o ciclo de vida do programa, coordenando as ações e distribuindo tarefas para os diversos sub módulos.

Na camada subsequente, denominada de módulos internos, é a camada de execução, onde componentes em paralelo lidam com tarefas específicas e independentes:

State: Mantém e fornece a lista de *peers* (nós da rede) especificamente para a interface de comando.

Peer_table: Faz o controle de estados e o armazenamento seguro de dados locais e conexões.

Rendezvous_connection: Encarregado da comunicação de descoberta, executando protocolos como *REGISTER*, *DISCOVER* e *UNREGISTER*.

Keep_alive: Garante a saúde da rede, enviando sinais de *PING/PONG* e medindo a latência através do *RTT* (Round-Trip Time).

Message_router: Atua como o encaminhador do sistema, realizando o roteamento adequado de mensagens.

Peer_server: Gerencia a infraestrutura de rede direta, lidando com conexões TCP e os processos de *Handshake*.

CLI: A interface de linha de comando que traduz as ações do usuário em comandos para o sistema.

A camada final representa as fronteiras do sistema, mostrando como os módulos internos se comunicam com o mundo exterior. O sistema interage com o Servidor Rendezvous (para registro e descoberta), com Outros Peers na Rede (conexão direta via TCP) e com o Usuário do Terminal (que opera o software via CLI).

1.1 DEPENDÊNCIAS E BIBLIOTECAS ESSENCIAIS

Para esse projeto que demanda comunicação em redes, de maneira impressionante, o sistema foi projetado de forma que não foi necessário o de dependências externas complexas (ou até mesmo algumas como *socket*, *threads*...).

Asyncio:

Em termos, é a base de todos os procedimentos do sistema. De modo a ser um nó de comunicação P2P, o cliente precisa realizar múltiplas tarefas simultaneamente: escutar o teclado do usuário na CLI, aguardar conexões TCP no servidor e enviar *pings* de manutenção. O *asyncio* permite que todas essas operações de I/O ocorram de forma concorrente e não bloqueante, sem a complexidade excessiva de gerenciar múltiplas *threads* tradicionais.

Logging:

Biblioteca essencial para criar o sistema de monitoramento duplo do cliente, permitindo gravar um histórico detalhado em arquivo enquanto aplica filtros dinâmicos para exibir apenas informações limpas e relevantes na interface do terminal. Possibilita a impressão de logs no terminal dos mais diversos tipos (WARNING, ERROR, INFO...), para um debug mais eficiente.

Argparse e json:

Trabalhando em conjunto no arquivo de inicialização, o *json* permite a persistência de configurações padronizadas em disco, enquanto o *argparse* captura e tipifica os parâmetros passados em tempo real na linha de comando, permitindo sobrescrever regras dinamicamente na inicialização do nó.

Pathlib e dataclasses:

O *pathlib* oferece uma interface segura para a manipulação dos caminhos dos arquivos de configuração, garantindo compatibilidade entre diferentes sistemas operacionais. Já o uso de *dataclasses* (como visto na tipagem das configurações) garante uma validação rígida de quais parâmetros o sistema aceita, evitando que chaves desconhecidas quebrem o funcionamento da rede.

1.2 PASSOS NECESSÁRIOS PARA EXECUTAR PROGRAMA EM LINUX

Para executar o programa de Chat P2P, os requisitos mínimos são:

- 1) Possui um sistema operacional de distribuição Linux moderna (como Ubuntu ou Debian).
- 2) Interpretador de linguagem Python 3.x instalado, em que recomenda-se Python 3.10 ou superior, dado o uso extensivo da biblioteca *asyncio* e de funcionalidades como *type hinting* e *dataclasses*, de forma a garantir-se compatibilidade total.
- 3) Acesso à Internet (especificamente à porta **8080** do servidor Rendezvous) e permissão para abrir sockets TCP locais (porta padrão **8081**).

Uma vez que o projeto foi desenvolvido em Python, não é necessária compilação. Além disso, o projeto utiliza predominantemente a biblioteca padrão do Python, o que significa que não é necessário instalar pacotes externos via *pip*.

Por fim, para executar o programa, primeiro deve-se pelo terminal entrar na pasta que contém o projeto, até a pasta `./chat`, que contém todos os arquivos com os códigos. Primeiro é realizado um teste de integridade no terminal através de:

```
> python3 -c "import peer_table; import state; import rendezvous_connection; import keep_alive; import message_router; import peer_connection; import cli; import p2p_client; import main; print('todos os módulos importam sem erro')"
```

Tenta-se assim importar, um por um, os módulos principais definidos na arquitetura do seu projeto, e caso haja sucesso imprime 'todos os módulos importam sem erro' no terminal. Em seguida, executa-se:

```
> python3 main.py --config config.json --name (inserir nome)
--namespace (inserir namespace) --log-file (inserir arquivo de
logs)
```

Com isso, cria-se oficialmente o nosso peer de identidade `name@namespace`, define-se em qual arquivo devem ser impressos os logs do `FileHandler`, e executa-se `main.py`, que desencadeia toda a execução do projeto de ChatP2P.

2.0 DESENVOLVIMENTO DO CÓDIGO

Com a estrutura da arquitetura definida, é hora de entrar no detalhamento técnico dos componentes. Cada arquivo do sistema foi projetado sob o princípio da responsabilidade única, contendo funções específicas para manipular o estado da rede, rotear pacotes ou gerenciar conexões físicas.

Inicialização: *main.py*

O arquivo `main.py` atua como o ponto de entrada principal e o gestor de configurações do cliente P2P. A execução do programa se inicia na função ***main()***, que serve como uma ponte entre o ambiente síncrono do Python e a lógica assíncrona da aplicação. Ela utiliza ***asyncio.run()*** para disparar a função central de preparação, além de capturar interrupções manuais do usuário para garantir que o sistema retorne o código de saída correto e seja encerrado adequadamente. O coração desse processo de preparação é a função assíncrona ***initializer()***, que orquestra a leitura e a validação dos parâmetros de funcionamento do nó P2P.

O sistema coleta os argumentos passados no terminal através da função ***get_config_args()*** e os combina com os dados de um arquivo estático (geralmente `config.json`) lido pela função ***loadfile()***. A lógica de prioridade entre essas duas fontes é resolvida pela função ***merging()***, que assegura que as opções digitadas no terminal sobrescrevam as configurações do arquivo de texto, além de adaptar a formatação dos parâmetros substituindo hifens por underscores. Em seguida, o ***initializer()*** ignora configurações desconhecidas, aplica valores padrão a variáveis críticas que possam estar faltando (como `hosts` e intervalos de descoberta).

Temos também o sistema de monitoramento de eventos, gerenciado pela função ***setup_log()***. Ela é responsável por configurar o nível de verbosidade do sistema e pode dividir o fluxo de informações de forma inteligente. Se a gravação em arquivo estiver ativada, a função cria um histórico persistente, registrando cada evento com carimbos

temporais. Simultaneamente, ela aplica um filtro customizado chamado **CLIFilter** para a saída de texto no terminal. Esse filtro atua como uma barreira visual, garantindo que o usuário veja na tela apenas as mensagens essenciais e marcadas com as tags **[CLI]**, **[P2P]**, **[MSG]** ou **[PUB]**, evitando assim que a interface do chat seja poluída com dados técnicos de infraestrutura. Caso a gravação em arquivo seja desabilitada, o comportamento muda e todos os eventos, independentemente da tag, passam a ser exibidos diretamente no console.

Gerenciador de métodos: *p2p_client.py*

O arquivo *p2p_client.py* funciona como o núcleo centralizado de toda a aplicação, sendo classificado na arquitetura como o Orquestrador Principal do sistema. Ele é responsável por traduzir as configurações consolidadas no momento da inicialização em instâncias operacionais concretas, utilizando para isso uma estrutura de dados rígida e validada através da classe **ConfiguracoesJson**. A sua principal entidade, a classe **p2pChatApp**, assume o papel vital de instanciar, interconectar e coordenar todos os submódulos internos do ecossistema P2P. Isso engloba desde a manutenção de tabelas locais com o **peer_table** até a ativação dos mecanismos de roteamento com o **message_router**, os testes de latência assíncronos do **keep_alive**, os protocolos de descoberta do **rendezvous_connection** e a própria interface de linha de comando (CLI).

O controle é centralizado dentro do seu método assíncrono **run()**. Ao ser disparado pelo ponto de entrada do programa, este método é encarregado de colocar toda a malha de rede em movimento de forma simultânea e coordenada através do **asyncio**. Ele inicia o servidor TCP (**peer_server**) para escutar e aceitar conexões de entrada de outros nós da rede, ativa as tarefas em segundo plano para a manutenção e monitoramento periódico da saúde das conexões, e dispara a execução contínua do prompt de comando para o usuário. Além de coordenar a inicialização e a concorrência dessas rotinas assíncronas, o *p2p_client* também é o responsável por garantir um encerramento seguro do nó.

Descoberta em rede via IP privado:

rendezvous_connection.py

rendezvous_connection.py é o ponto de entrada do nó na rede P2P. A sua função é direta e vital: estabelecer comunicação contínua com o IP fixo e a porta do Servidor Rendezvous (cujos valores são carregados na inicialização do sistema). Como os clientes em uma rede descentralizada entram e saem a todo momento com endereços de IP dinâmicos, este servidor atua como a única âncora estática e imutável da infraestrutura,

servindo como uma lista telefônica central para que os nós consigam se encontrar na internet.

O funcionamento do módulo baseia-se na execução sequencial de três operações fundamentais de descoberta:

- **REGISTER:** Logo após a inicialização, o cliente envia os seus dados de roteamento (IP atual, porta de escuta TCP e o seu *namespace*) para o IP fixo do servidor central, anunciando publicamente a sua entrada na malha.
- **DISCOVER:** De forma periódica, o módulo faz requisições ao servidor para baixar a lista mais recente de pares ativos. São esses dados que alimentam o estado interno do cliente, permitindo que o sistema saiba quem está online para estabelecer conexões diretas.
- **UNREGISTER:** Durante o processo de desligamento (via */quit*), o módulo envia uma notificação final ao servidor central pedindo a remoção imediata do seu registro. Isso garante uma saída limpa e evita que o nó fique como um contato "fantasma", o que causaria falhas de conexão para o resto da rede.

Conexões entre *peers*: *peer_connection.py*

Após a consulta ao servidor, registro do usuário e coleta de IPs dinâmicos, *peer_connection.py* é o verdadeiro motor de rede P2P da aplicação. Enquanto o módulo de descoberta lida com metadados e o servidor central, o *peer_connection* é responsável por criar, manter e gerenciar os túneis de comunicação diretos (via protocolo TCP) entre o seu computador e os outros usuários da malha. Em uma arquitetura descentralizada, este módulo assume uma dupla personalidade vital: ele atua simultaneamente como servidor (escutando passivamente quem quer se conectar a ele) e como cliente (iniciando ativamente conexões com os nós recém-descobertos). O *peer_connection* age imediatamente após o *rendezvous_connection*. Assim que o nó baixa a "lista telefônica" do servidor central, o *peer_connection* entra em ação para transformar esses nomes em conexões de rede reais.

Para que a rede descentralizada funcione, o módulo atua simultaneamente de duas formas:

- **Atuação como Servidor (Tráfego *Inbound*):**
 - **Ação:** Mantém um **socket** passivo aberto (na porta definida em *main.py*), escutando a rede.

- **Segurança (Handshake):** Ao aceitar uma conexão, força um "aperto de mão". Isso obriga a máquina remota a provar sua identidade (*peer_id*), barrando conexões de intrusos ou de softwares alheios ao chat.
- **Atuação como Cliente (Tráfego Outbound):**
 - **Ação:** Funciona de forma ativa. Assim que a aplicação descobre um novo usuário válido na rede, este módulo cria um *socket* de cliente e toma a iniciativa de se conectar a ele, abrindo uma rota de saída.

Tabela de roteamento e estado: *peer_table.py*

Visando os problemas iminentes de deixar variáveis soltas, o módulo *peer_table.py* encapsula os dados de cada usuário em registros consolidados. Para cada nó descoberto, ele memoriza:

- **Identidade e Localização:** O *peer_id* (quem é o nó) e o seu namespace (a qual região, sala ou grupo lógico ele pertence).
- **Métricas de Saúde:** Armazena o *rtt_ms* (Round-Trip Time, que é a latência da conexão) e o *connection_state* (se o nó está ativo, pendente ou marcado como STALE).

Daí, o *peer_table* atua como o banco de dados em memória e a "lista de endereços" do P2P. Ele centraliza o armazenamento de todas as informações sobre os outros participantes da rede, garantindo que os demais módulos saibam exatamente para quem e como enviar dados. A interface de linha de comando *CLI*, por exemplo, consome essas informações diretamente para exibi-las na tela quando o usuário executa comandos de consulta, como o */peers* ou o */rtt*. O roteador *message_router* também depende intimamente desta estrutura, consultando a tabela antes de despachar qualquer pacote para validar se o destinatário realmente existe e qual é o seu contexto na rede.

Ainda, *peer_table* possui métodos específicos para agrupar e filtrar os nós com base em seus *namespaces*. Isso permite que a sua rede P2P suporte subdivisões lógicas, garantindo que o sistema não seja apenas um canal de comunicação global, mas consiga direcionar mensagens em massa de forma seletiva apenas para os usuários que compartilhem o mesmo ambiente.

Monitoramento da Rede: *keep_alive.py*

Enquanto o servidor estabelece os túneis físicos e o roteador despacha as mensagens do usuário, o componente *keep_alive* opera de forma invisível e assíncrona em

segundo plano. Esse deve monitorar proativamente a malha de rede, garantindo que as vias de comunicação permaneçam ativas, saudáveis e responsivas sem a necessidade de qualquer intervenção manual.

O funcionamento central baseia-se em um ciclo contínuo de sondagem. Respeitando o intervalo temporal definido nas configurações de inicialização, o sistema dispara pacotes de controle do tipo **PING** para todos os pares com os quais o nó possui uma conexão TCP aberta, registrando o carimbo de tempo (*timestamp*) exato do envio. Quando o nó remoto recebe a solicitação e devolve um pacote de confirmação **PONG**, o módulo intercepta esse retorno e calcula matematicamente a diferença temporal. O resultado é o **RTT** (*Round-Trip Time* ou Tempo de Ida e Volta), que reflete com precisão a latência da conexão em milissegundos.

Paralelamente, o mesmo descarrega os resultados do **RTT** diretamente no **peer_table**, atualizando a latência de cada contato em tempo real. Se um nó vizinho ficar instável e não responder aos chamados dentro de uma janela de tolerância aceitável, este módulo altera o status desse par para **STALE** na tabela. Essa marcação de inatividade atua como um gatilho de segurança crítico: ela alerta o roteador para abortar o envio de novas mensagens para aquele destino e sinaliza ao gestor de rede que os sockets ociosos daquele usuário já podem ser destruídos, economizando largura de banda e processamento da máquina.

Roteamento de mensagens: *message_router.py*

O *message_router.py* define a classe **MessageRouter**, atuando como o redistribuidor assíncrono do sistema distribuído. Enquanto o servidor lida com os túneis físicos, este módulo utiliza a biblioteca **asyncio** para orquestrar o envio de pacotes de forma não bloqueante e a biblioteca **uuid** para gerar identificadores únicos e infalíveis para cada mensagem que trafega na malha.

O estado base do roteador é definido logo na sua inicialização, através do construtor **__init__**, que recebe as dependências essenciais do sistema: a tabela de estado **state**, o gestor de conexões **peer_server**, o sistema de logs e um valor de Time-To-Live (**tll** padrão de 1) para evitar que pacotes fiquem vagando infinitamente pela rede. A ativação e o encerramento seguro de suas operações em segundo plano são controlados, respectivamente, pelos métodos assíncronos **start()** e **stop()**.

Quando o usuário decide se comunicar, o fluxo passa pelo método principal **send()**. ela recebe os parâmetros da interface (**peer_id**, **namespace**, o corpo da **message**, o **mode** e a exigência de confirmação via **require_ack**) e constrói o pacote final, enviando-o para um ou mais nós de destino.

Além de gerenciar o tráfego de dados, o roteador atua na resiliência da rede através do método ***trigger_reconcile()***. Se a topologia sofrer instabilidades, este método é acionado para tentar reparar as conexões. Ele verifica se a instância do roteador possui acesso ao objeto principal ***chat_app*** e, caso positivo, invoca a função ***reconcile_after_discover()*** para forçar a aplicação a remapear e reconectar com os pares perdidos.

Infraestrutura da rede: ***peer_connection.py***

Ele é o responsável direto por gerenciar a malha física de conexões estabelecidas entre o nó local e todos os outros pares integrados através do servidor *rendezvous*.

Para manter o código organizado e eficiente, o módulo divide as suas responsabilidades de rede em duas estruturas principais:

Entidades de Dados e Controle

- **Data Class ConnectionInfo:** Funciona como um container limpo e padronizado. A sua função é abstrair e organizar os metadados (como IP, porta e direção do tráfego) e o estado atual de cada conexão individual ativa no sistema.
- **Classe PeerServer:** É o verdadeiro núcleo de comunicação de rede do nó P2P. A sua grande sofisticação técnica reside na capacidade de possuir uma arquitetura de dupla personalidade:
 - **Servidor (Inbound):** Escuta a rede de forma passiva, validando e aceitando conexões de entrada de outros usuários.
 - **Cliente (Outbound):** Atua de forma proativa, solicitando e abrindo rotas de saída em direção a pares recém-descobertos.

Durante a sua inicialização, o construtor recebe as coordenadas locais (***host, port***) e guarda as referências dos componentes globais (***state, peer_table, logger***). É neste momento que ele estrutura os atributos vitais de governança interna:

- ***self.connections:*** Um dicionário vital que mapeia o identificador do nó remoto (***peer_id***) ao seu respectivo objeto *ConnectionInfo*. Atua como a tabela viva de sockets ativos na malha.
- ***self.dialing:*** Um set (conjunto) temporário usado como trava de segurança. Ele armazena quem está sendo chamado no momento, evitando que o cliente dispare conexões duplicadas ou em paralelo para o mesmo destino.
- ***self.set_for_tasks:*** Um rastreador essencial que cataloga todas as tarefas assíncronas em execução no background. É ele que garante um desligamento limpo (*graceful shutdown*) do cliente, impedindo processos fantasmas.

- ***self._server***: Variável que guarda a instância concreta do servidor gerado pelo *asyncio*.
- ***self._running***: Uma variável booleana simples, o qual determina se o servidor deve continuar operando ou iniciar o encerramento seguro.

Lista dos peers para CLI: *state.py*

Na função principal de estabelecer a ponte com o usuário, o módulo *State* atua como um repositório de dados simplificado, focado inteiramente na interface de linha de comando (**CLI**). A sua responsabilidade primária é atuar como a "cara" do sistema: ele formata, filtra e disponibiliza os dados dos usuários conectados de maneira imediata. Isso garante que o terminal exiba informações limpas e compreensíveis sempre que o usuário solicitar uma listagem de nós ativos na malha.

O design do sistema aplica um forte princípio ao separar claramente este módulo do *peer_table*.

- **O Motor (*peer_table*)**: Assume o trabalho pesado da infraestrutura, gerenciando métricas precisas de latência (*rtt*), pings do *keep_alive* e estados de conexão a nível de socket (*STALE*, *pendente*).
- **A Visão do Usuário (*state*)**: Encapsula e abstrai esses detalhes de baixo nível. Ele foca apenas em entregar as informações legíveis para a experiência do usuário, livrando a **CLI** da necessidade de interpretar dados complexos de rede.

Logo na fase de inicialização comandada pelo *p2p_client*, o objeto *state* é criado e injetado como dependência nos módulos operacionais (como o *peer_server*). À medida que o servidor descobre novos pares e consolida túneis físicos, o estado interno é atualizado continuamente de forma assíncrona. Consequentemente, no exato milissegundo em que o usuário digita um comando de consulta, o terminal consome uma fotografia fidedigna e em tempo real da topologia da rede descentralizada.

3.0 CONCLUSÃO

O desenvolvimento deste cliente P2P demonstra a viabilidade de construir uma rede descentralizada robusta a partir do zero. A força da arquitetura reside na sua alta modularidade, separando a interface, o roteamento e a infraestrutura física.

Ao explorar os recursos do *asyncio*, o sistema consegue gerenciar conexões concorrentes, testes de latência e a troca de mensagens de forma simultânea e assíncrona, mantendo a aplicação leve e livre de dependências externas complexas. O projeto vai além

de um simples aplicativo de chat: ele entrega uma malha de rede resiliente e autônoma, consolidando os princípios do modelo Peer-to-Peer e devolvendo aos usuários o controle sobre as suas comunicações.